

Documentazione di Progetto: Software Architecture Design

Task R6: Gestione Profilo Giocatore

0. Identificazione del Team e Obiettivi

Campo	Dettaglio
ID Team	Gruppo D3
Componenti	Andrea Pironti, Antonio Coppola, Mario Berrino
Task Assegnato	Task R6, Gestione profilo giocatore
Approccio	Sviluppo Incrementale
Repository Originale	https://github.com/Testing-Game-SAD-2023/A13
Repository Consegnato	https://github.com/antoniocoppola0119-lgtm/A13/tree/main

1. Contesto del Progetto

TestingRobotChallenge è una piattaforma di gamification per l'apprendimento del software testing. Il sistema permette di sfidare avversari automatizzati ("Robot") scrivendo test JUnit4. L'obiettivo del **Task R6** è l'evoluzione dell'architettura esistente tramite la trasformazione della pagina profilo utente, aggiornandone logica e UI.

2. Analisi dei Requisiti

Requisiti Funzionali (RF)

Codice	Nome	Descrizione
RF1	Visualizzazione Informazioni Fisse	Il sistema deve mostrare nella pagina profilo le informazioni statiche del giocatore, includendo nome, foto/avatar e biografia.
RF2	Selezione Contenuti Dinamici	L'utente deve poter selezionare, tramite un selettore dedicato, la categoria di informazioni da visualizzare nella sezione dinamica del profilo.
RF3	Visualizzazione Achievement	Il sistema deve integrare la visualizzazione degli achievement tra le opzioni selezionabili.
RF4	Gestione Persistenza Profilo	Il sistema deve consentire il salvataggio e il recupero di biografia, nickname e avatar tramite il microservizio T23.
RF5	Navigazione Modulare	Il passaggio tra le sezioni del profilo deve avvenire senza ricaricare l'intera pagina.

Codice	Nome	Descrizione
RF6	Visualizzazione Progressi Giocatore	L'utente deve poter visualizzare livello, rango e barra XP.
RF7	Visualizzazione Partite Passate	L'utente deve poter consultare lo storico delle partite con risultati, achievement e dati degli avversari.
RF8	Gestione Social	L'utente deve poter gestire follower/following, cercare giocatori e seguire o smettere di seguire utenti.

Requisiti Non Funzionali (RNF)

Codice	Nome	Descrizione
RNF1	Estensibilità	L'architettura deve permettere l'aggiunta futura di nuove sezioni senza modificare la struttura principale.
RNF2	Usabilità	L'interfaccia deve evitare sovraccarico informativo e garantire una navigazione intuitiva.
RNF3	Coerenza	Lo stile della pagina profilo deve essere coerente con la pagina Achievement.
RNF4	Manutenibilità	I modelli dati del microservizio T23 devono rispettare le convenzioni esistenti.
RNF5	Prestazioni	Il caricamento delle sezioni deve avvenire con latenza minima ottimizzando le chiamate API.

Use Case / Scenari / User Stories

Use Case / Scenari

- **Gestione Identità**

Il giocatore può gestire il proprio profilo tramite il caso d'uso *Modifica Profilo*, che include specificamente la *Modifica immagine profilo* e la *Modifica bio*.

- **Visualizzazione Progressi (Rank)**

All'interno del *RankContainer*, il sistema permette di:

- Visualizzare la barra esperienza (con dettaglio sui punti mancanti)
- Visualizzare il grado attuale con relativa icona
- Consultare i dettagli di tutti i ranghi tramite un *Modal* informativo

- **Interazione Social**

L'utente può visualizzare le liste *Follows/Followed* ed effettuare ricerche tra i contatti.

- **Cronologia Partite**

Tramite il selettore, l'utente accede alla *Visualizzazione ultime partite giocate*.

User Stories

- **Bio e Avatar**

In qualità di *Giocatore*, voglio poter inserire una breve biografia, nickname e un'immagine del profilo

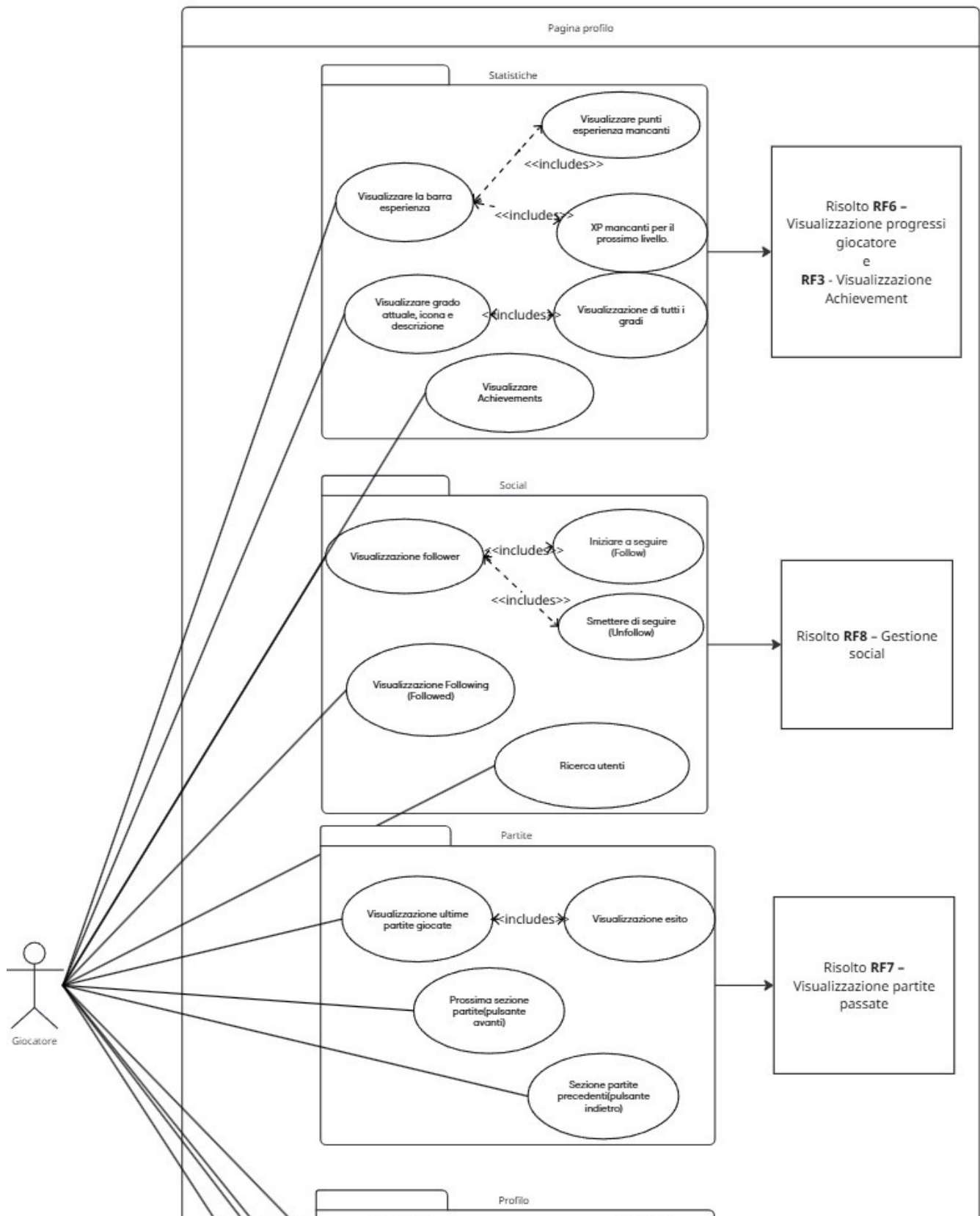
affinché gli altri utenti possano identificarmi.

- **Sistema di Ranghi**

In qualità di *Giocatore*, voglio visualizzare il mio grado attuale e la barra di avanzamento per monitorare i miei progressi nel gioco.

- **Navigazione Dinamica**

In qualità di *Giocatore*, voglio usare un selettore per passare dalla visualizzazione delle mie partite alle informazioni social senza cambiare pagina.



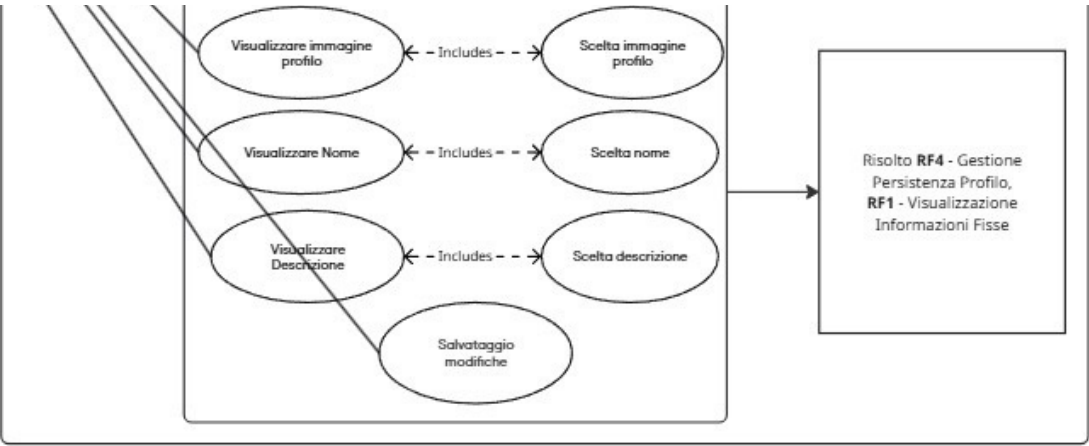


Figura 1: Use Case diagram che descrive le interazioni del Giocatore con i moduli Statistiche, Social, Partite e Profilo per la gestione completa della propria identità e dei progressi, mostrando come si siano affrontati i vari requisiti.

Categoria	Modulo	Modifica
Aggiunti (Frontend)	profileMain.js	Punto di ingresso principale: gestisce l'inizializzazione dei listener e lo switch dinamico delle view (Social, Matches, Stats) senza ricaricare la pagina.
Aggiunti (Frontend)	profileFixed.js	implementa la logica di editing per Bio e Nickname con limiti di caratteri (20/200) e la selezione dell'avatar.
Aggiunti (Frontend)	stats.js	gestisce la mappatura degli XP nei dieci Ranghi, l'aggiornamento della barra di progresso e il rendering degli achievement.
Aggiunti (Frontend)	social.js	gestisce le chiamate per follower/following, la logica di ricerca utenti e l'aggiornamento dello stato del follow.
Aggiunti (Frontend)	matches.js	recupera la storia dei match, gestendo la paginazione lato client e la visualizzazione dei badge vittoria/sconfitta.
Aggiunti (Backend)	GameProgressService (T23)	implementa la logica per il recupero dello storico completo delle partite di un giocatore tramite getGameHistoryByPlayer. Utilizza il MapperFacade per restituire una lista di GameProgressDTO al microservizio T5.
Modificati (Backend)	UserProfileController (T5)	aggiunte rotte utili per la logica della pagina profilo
Modificati (Backend)	T23Service (T5)	aggiunte servizi utili per la logica della pagina profilo
Modificati (Backend)	UserGameProgress (T5)	aggiunto attributo "Achievements"
Modificati (Backend)	UserSocialController (T23)	aggiunte rotte per editare profilo e cercare profili utenti tramite termini di ricerca
Modificati (Backend)	UserSocialService (T23)	aggiunti servizi per editare profilo e cercare profili utenti tramite termini di ricerca

Categoria	Modulo	Modifica
Rimossi dalla logica ma non dal progetto (Frontend)	Achievements.html (T5)	Vecchia pagina degli obiettivi; i contenuti sono ora integrati dinamicamente nel nuovo profilo.
Rimossi dalla logica ma non dal progetto (Frontend)	Profile-Edit.html (T5)	Vecchia pagina di modifica; sostituita dalla modalità di editing "in-place" gestita da profileFixed.js
Rimossi dalla logica ma non dal progetto (Frontend)	GameHistory.html (T5)	Vecchia pagina dello storico partite; i contenuti sono ora integrati dinamicamente nel nuovo profilo.
Rimossi dalla logica ma non dal progetto (Backend)	UserProfileController (T5)	rotta per /Achievements deprecata.
Rimossi dalla logica ma non dal progetto (Backend)	UserProfileController (T5)	rotta per /Edit-Profile deprecata.
Rimossi dalla logica ma non dal progetto (Backend)	UserProfileController (T5)	rotta per /GameHistory deprecata.
Da implementare (sviluppi futuri)	Notifiche Profilo	Da implementare le notifiche sulla pagine /profile.
Da implementare (sviluppi futuri)	Immagini Personalizzate	La scelta degli avatar è limitata a una lista fissa; l'upload di file personalizzati è rimandato.
Da implementare (sviluppi futuri)	Chat tra utenti	Chat tra utenti che si seguono ancora da implementare.

3. Progettazione della Soluzione

Decisioni architetturali adottate

- **Separazione delle Responsabilità (Separation of Concerns)**

È stata mantenuta una netta distinzione tra la logica di presentazione (gestita in T5) e la logica di persistenza dei dati. I dati anagrafici di base e le varie informazioni specialistiche (social e statistiche) sono stati centralizzati in T23, garantendo un'architettura disaccoppiata.

- **Design Modulare e Dinamico della UI**

Seguendo le linee guida del task, si è deciso di implementare un layout ibrido: una sezione fissa per l'identità (Avatar/Bio) e una sezione variabile basata su un selettore. Questa scelta riduce il sovraffollamento informativo e permette di aggiungere nuovi moduli in futuro senza modificare la struttura portante della pagina.

- **Integrazione tramite API Gateway**

Tutte le comunicazioni tra il front-end (T5) e i servizi di backend (T23) sono state intermedie dall'API Gateway. Questa scelta architetturale permette di astrarre gli endpoint reali dei microservizi, fornendo un punto unico di accesso e facilitando la gestione delle politiche di sicurezza e routing.

Microservizi coinvolti e modificati

- T5
- T23

Code Smells e Anti-Patterns

- (T23) Tutte le chiamate che includevano la gestione della classe GameProgress andavano al PlayerProgressService, risolto aggiungendo la classe GameProgressService
- (T5-T23) Alcuni nomi delle rotte in T5 non combaciavano con quelli in T23, risolto modificando appropriatamente i nomi in T5 (specificatamente in T23Service)
- (T5-T23) Alcuni tipi di ritorno non combaciavano tra T5 e T23, risolto aggiungendo in T5 le conversioni appropriate.

4. Implementazione e Architettura

Il sistema utilizza una **Layered Architecture** distribuita tra il microservizio **T5** (UI) e il microservizio **T23** (Persistenza), comunicanti tramite REST.

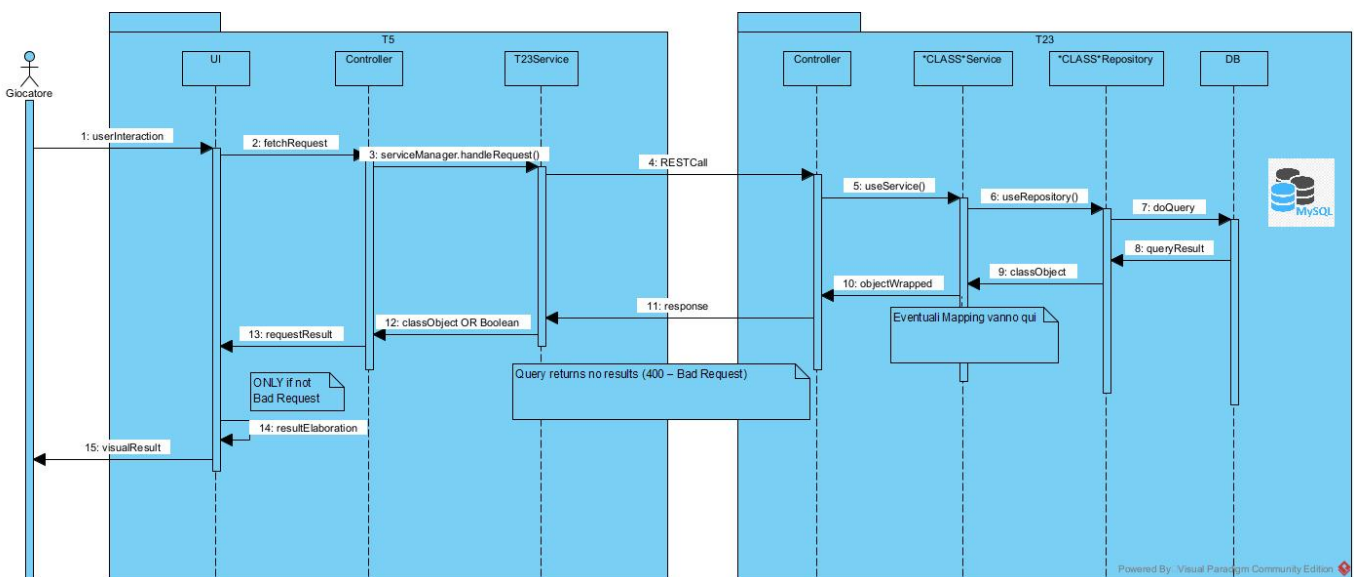


Figura 2: Sequence diagram che descrive le interazioni del Giocatore tramite l'UI con il backEnd, ogni iterazione segue un preciso protocollo

- **T5:** Sostituiti i vecchi file `Profile.js/html/css` per supportare il nuovo design, seguendo il modulo standard ES6.

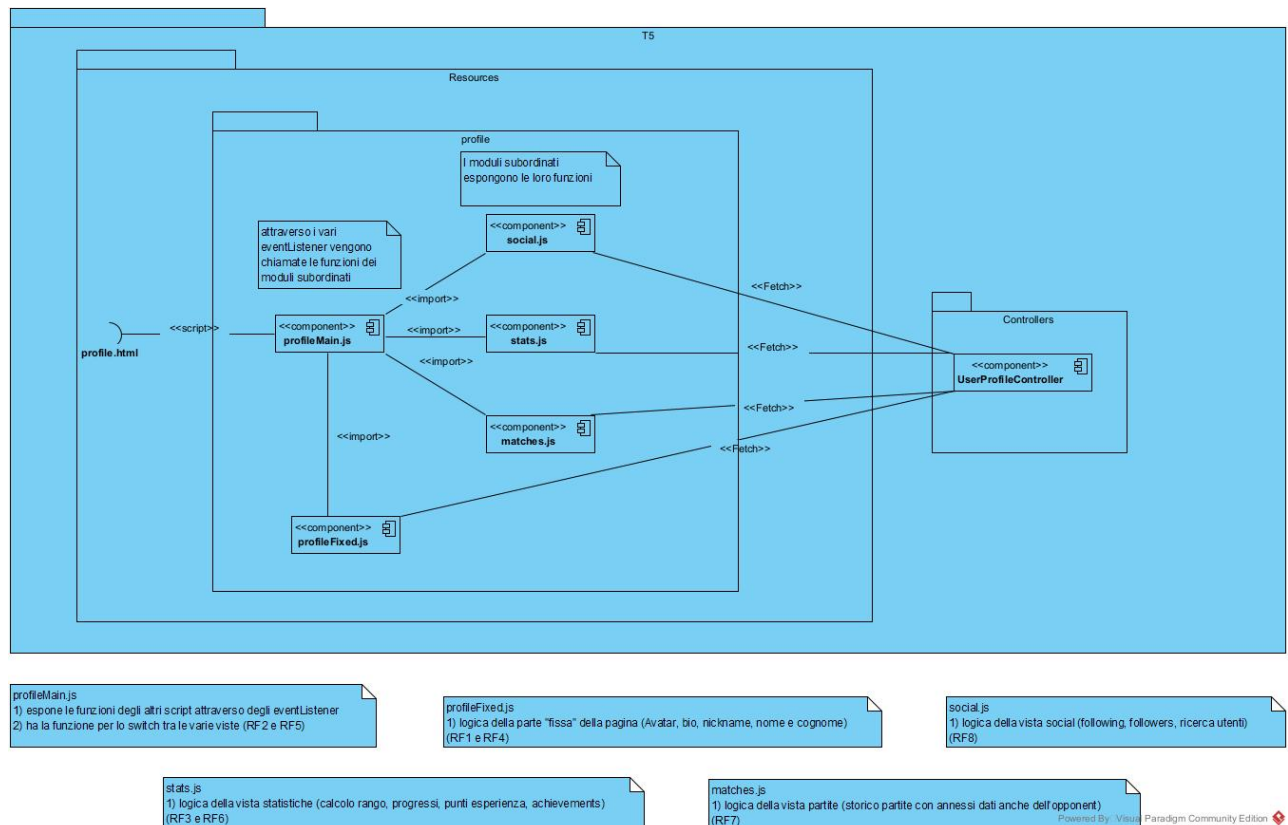


Figura 3: Component diagram che illustra l'architettura modulare del frontend in cui profileMain.js coordina i moduli subordinati per le chiamate asincrone verso il controller.

- **T23:** Potenziato con **GameProgressService** per la gestione dello storico.

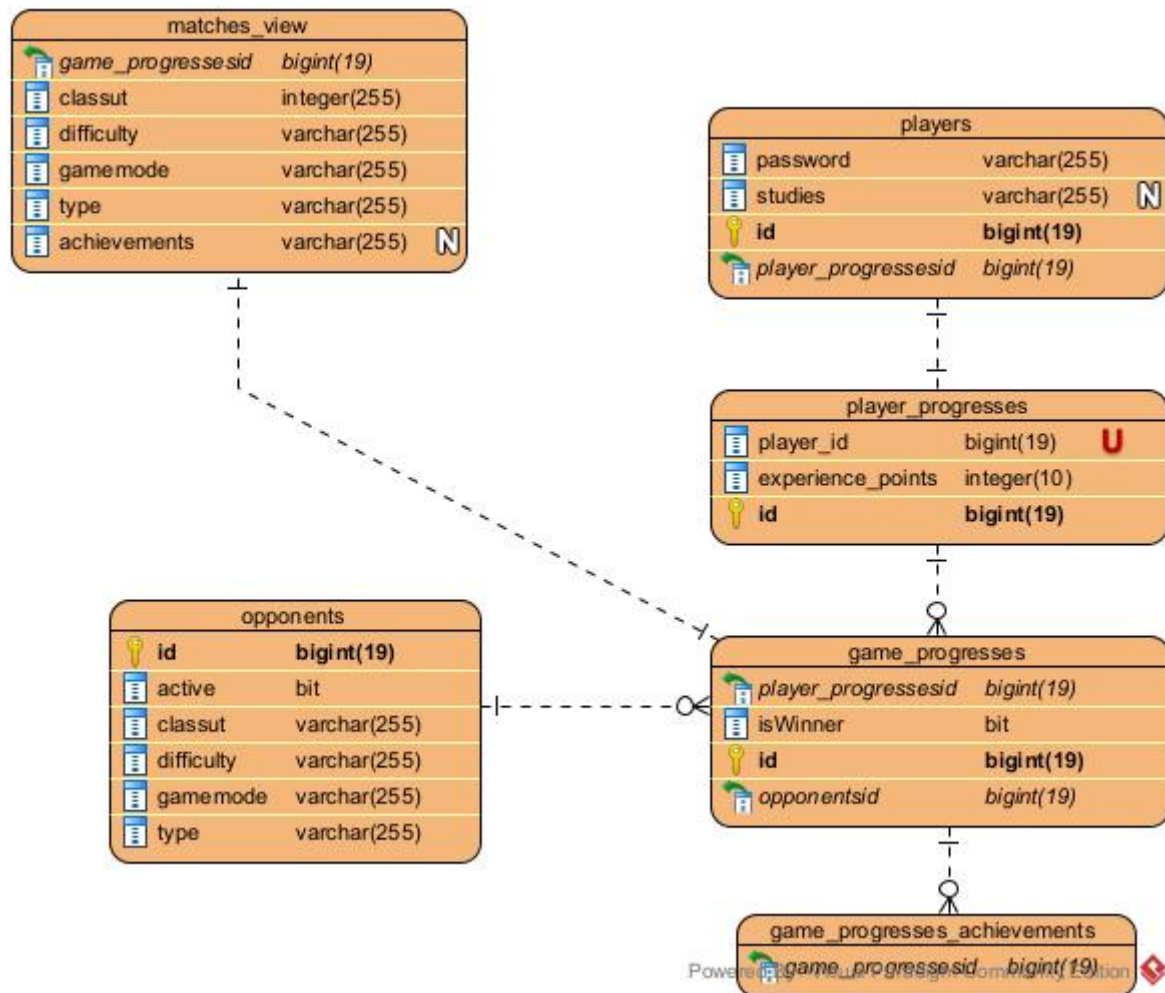
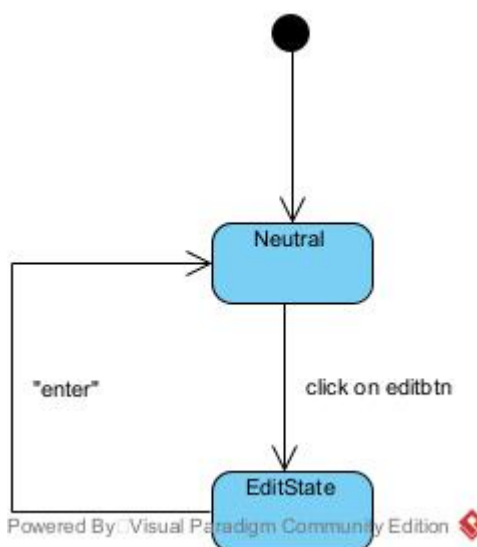


Figura 4: Class diagram che descrive la relazione tra cosa viene visto per ogni match nell'UI e le classi nel MySQL, a partire dall'id del giocatore si trovano i progressi, poi i game_progress e da lì si ottengono le informazioni utili dall'opponent e dagli achievements

- **Editing:** Implementata una macchina a stati per passare da "Neutral" a "EditState" tramite interazione dell'utente.



5. Testing

Tipologie di test effettuati

- User Interface Testing
- Static Analysis e Code Reading

Risultati ottenuti

- Pagina Web funzionante per modifica profilo, visualizzazione progressi utente e rank, visualizzazione partite utente, achievements, visualizzazione Followers/Following e ricerca generica giocatori con toggle follow.

Problemi rilevati e relative soluzioni

- Problemi grafici di interfaccia utente risolti modificando il file CSS.
- Problemi con il toggle follow (con il pulsante *segui* per utenti già seguiti, il codice rimuoveva l'utente dai seguiti). Risolto salvando una cache di follower dell'utente, facendo sì che il pulsante *segui* non uscisse per utenti già seguiti.
- Impossibilità ad accedere alla pagina profilo. Risolto collegando la chiamata `/profile` al nome e cognome del giocatore.
- Assenza di vincoli sulla lunghezza degli input utente (Nickname e Bio), con potenziale rischio di overflow nella UI. Risolta implementando vincoli lato client tramite attributo `maxlength` e validazione JavaScript.

6. Sviluppi Futuri e Raccomandazioni

Possibili estensioni future del progetto e raccomandazioni per il mantenimento

- Aggiungere le notifiche alla pagina profilo.
- Implementare una chat tra utenti nella pagina.
- Permettere all'utente di scegliere immagini personalizzate come avatar.
- Permettere all'utente di visualizzare i profili degli altri giocatori.

Appendice – Riferimenti

- **Link GitHub:** <https://github.com/antoniocoppola0119-lgtm/A13/tree/main>